

Integrating an External SOAP Partner

A Design for the CampusEats Platform

Reading and Authoring a WSDL, Forming SOAP Messages by Hand,
and Describing Discovery in Its Modern Form

Team ID: 7
Course: CS543, Web Services
Assignment: Assignment 3, Module 1
Reference Tutorial: Tutorial 3 of 14, Lecture 3
Instructor: Dr. Pramit Mazumdar
Repository: <https://github.com/aditi-garg-0/campuseats>
Date of Submission: August 21, 2026

Submitted by

Name	Roll Number
Aditi Garg	20251651008
Neha Nupur	20251651064
Shivam Kumar Soni	20251651084

This document constitutes the required `integration.pdf` for Assignment 3, addressing the Context, HTTP Binding, Discovery, and Fault Mapping tasks. The accompanying contract and message artefacts, namely `partner.wsdl`, `soap-request.xml`, `soap-response.xml`, and `soap-fault.xml`, are committed alongside this document in the repository referenced above.

1. Context: The Partner Edge

Partner: AasaanPay Secure Gateway, an external payment processor serving the campus bank

Operation Integrated: `charge`, used to authorise and capture a student's card payment

Consumer: The CampusEats *Orders* service, invoked at the moment a student confirms checkout

CampusEats is composed of six cooperating services, namely Accounts, Catalogue, Orders, Payments, Delivery, and Notifications, and every one of these is shaped around REST, since the CampusEats team owns both ends of each such contract and therefore prefers calls that are simple, cacheable, and oriented around resources. AasaanPay sits outside that boundary. It is a regulated financial system that CampusEats does not control, and it mandates the use of SOAP for three concrete reasons. First, a bank cannot tolerate an ambiguous request shape, and so AasaanPay publishes a rigid WSDL contract, one that a machine can validate, ruling out malformed charges long before they reach a ledger. Second, WS Security allows AasaanPay to sign and encrypt the SOAP envelope itself, so that a charge remains protected from end to end across any intermediary, and not merely across the transport layer. Third, the movement of money demands that a transaction either completes entirely or not at all, together with guaranteed delivery that cannot later be repudiated, and the WS star stack that accompanies SOAP is precisely what standardises this class of reliability. AasaanPay is therefore the single deliberate SOAP edge within an otherwise fully REST oriented CampusEats. It is integrated because the partner requires it, and not because the CampusEats team prefers it.

2. HTTP Binding: Carrying the SOAP Envelope

The `charge` envelope, presented in full in `soap-request.xml`, is carried as the body of a single synchronous HTTPS POST request. The method, host, content type, and `SOAPAction` shown below are copied verbatim from the `<soap:binding>` and `<soap:address>` elements of `partner.wsdl`, and the endpoint URL is precisely the one published under `service` and `port`.

```
POST /v2/payments HTTP/1.1
Host: gateway.aasaanpay.io
Content-Type: text/xml; charset=utf-8
Content-Length: <computed>
SOAPAction: "urn:aasaanpay:payments/charge"

<soap:Envelope> ... (see soap-request.xml) ... </soap:Envelope>
```

Reading the four fields against the WSDL. The `Host` value and the path `/v2/payments` together originate from `<soap:address location="https://gateway.aasaanpay.io/v2/payments"/>`, declared under `service AasaanPayPaymentsService`, port `AasaanPayPaymentsSOAP`. The `SOAPAction` value is copied from `<soap:operation soapAction="urn:aasaanpay:payments/charge"/>`, found inside the `charge` operation of binding `AasaanPayPaymentsSOAP`. The header `Content-Type: text/xml` reflects the document literal SOAP 1.1 style declared in `<soap:binding style="document"/>`. The call itself is synchronous: the *Orders* service blocks on this request and marks the order as paid only once a `chargeResponse`, or alternatively a `soap:Fault`, has been received.

3. Discovery in Its Modern Form

3.1 How CampusEats Obtains the WSDL

In practice, the onboarding team at AasaanPay emails the integrating merchant a direct, versioned link, namely `https://gateway.aasaanpay.io/wsdl/v2/Payments.wsdl`, and this remains, by a wide margin, the more common path taken in real partner integrations encountered in practice: it is simple, it is unambiguous, and it requires no additional infrastructure. The alternative path, mediated through a registry, is what UDDI formalised in its original conception. Under that model, the CampusEats integration layer would instead query a catalogue asking, in effect, who offers a payments service compatible with `urn:aasaanpay:payments`, and would receive in return an entry pointing at the same WSDL. Both paths are documented here, although the direct URL is what the `Orders` service actually binds to in practice. The registry entry given below is the catalogue record that would support a discovery based upon lookup, and is what an audit of partner onboarding would be expected to consult.

3.2 Registry and Catalogue Entry

Business	AasaanPay Financial Services Ltd.
Service	AasaanPayPaymentsService
Endpoint	<code>https://gateway.aasaanpay.io/v2/payments</code>
Specification (tModel)	Speaks <code>urn:aasaanpay:payments</code> ; WSDL published at <code>https://gateway.aasaanpay.io/wsdl/v2/Payments.wsdl</code>
Operations Exposed	<code>charge</code> , <code>refund</code>
Registered By	The CampusEats integration team, at the time of partner onboarding

Following this initial sequence of publish, then find, then bind, the registry withdraws entirely from the runtime path. Every subsequent `charge` call thereafter is a direct SOAP round trip between the CampusEats `Orders` service and the bound AasaanPay endpoint, with no registry lookup occurring on the path taken by live traffic.

4. Fault Mapping

AasaanPay reports a charge that has been declined, or a gateway that is unavailable, as a structured `soap:Fault`, shown in full in `soap-fault.xml`, rather than as a broken HTTP response. The fault therefore still arrives as a properly formed SOAP body that the `Orders` service can parse in a deterministic manner. The Assignment 2 `Payments` contract already promises callers of `placeOrder` a stable, CampusEats native error, namely “payment declined”, regardless of which processor happens to sit behind the scenes. The `Orders` service is therefore the boundary at which translation occurs: it inspects `soap:Fault/detail/pb:error/@code` and rewrites AasaanPay’s own vocabulary into the vocabulary CampusEats has promised, before anything reaches a student.

AasaanPay Fault Code		CampusEats Error	Surfaced to the Student
TXN_DECLINED	→	payment_declined	“Payment declined”
GATEWAY_TIMEOUT	→	payment_unavailable	“Payment declined” (may be retried)

This mapping is deliberate. Should AasaanPay itself later be replaced by a different bank, the `placeOrder` contract, together with every error string it promises to students, remains unchanged. AasaanPay’s internal codes, its namespace, and its WS Security header are never permitted to leak past the *Orders* boundary.

Accompanying artefacts committed alongside this document within the repository: `partner.wsdl`, `soap-request.xml`, `soap-response.xml`, and `soap-fault.xml`.